Algoritmos de Ordenamiento y Búsqueda I

Sesión 2 — CC112: Fundamentos de Programación

Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación
Universidad Nacional de Ingeniería

Tabla de Contenidos

1 Motivación: ¿por qué ordenar y buscar?

2 Bubble Sort — Ordenamiento por Burbuja

3 Selection Sort — Ordenamiento por Selección

4 Insertion Sort — Ordenamiento por Inserción

5 Búsqueda Lineal

6 Actividad práctica

¿Por qué son fundamentales el ordenamiento y la búsqueda?

- Un **directorio telefónico** está ordenado alfabéticamente para acelerar la búsqueda.
- Los **motores de búsqueda** ordenan millones de páginas por relevancia en milisegundos.
- En **estadística**: la mediana y percentiles requieren datos ordenados.
- **Machine learning**: k-NN, árboles de decisión y muchos algoritmos dependen de datos ordenados.

Sesión de hoy: algoritmos cuadráticos

Bubble Sort — burbuja

Selection Sort — selección

Insertion Sort — inserción

Búsqueda Lineal

Todos: complejidad $O(n^2)$ en el peor caso

Convenciones: arreglos de enteros, orden **ascendente**, índices desde 0.

Búsqueda en el mundo real

- Bases de datos de clientes en tiendas en línea. Detección de patrones en imágenes médicas. Verificación de identidad en sistemas de seguridad.

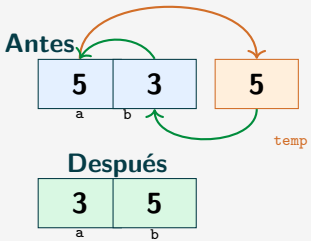
Operación clave: el intercambio (*swap*)

Todos los algoritmos de ordenamiento de esta sesión necesitan **intercambiar** dos elementos. Se requiere una variable temporal.

```
// Intercambio manual
void intercambio(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// C++ provee std::swap (recomendado)
swap(arr[i], arr[j]);
```

Visualización del intercambio



Bubble Sort — Idea central

Principio

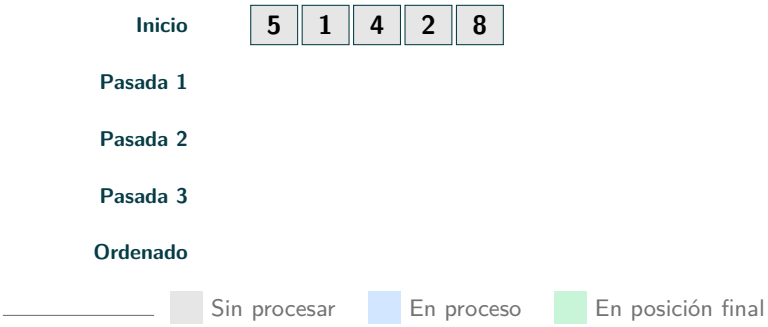
Recorre el arreglo comparando **pares adyacentes**. Si están en orden incorrecto, los **intercambia**. Al final de cada pasada, el **mayor elemento** "flota" hasta su posición final (como una burbuja).

- Para N elementos se necesitan como máximo $N - 1$ pasadas.
- En la pasada i, los últimos i elementos ya están en su lugar definitivo.
- **Optimización:** si en una pasada no hay ningún intercambio, el arreglo ya está ordenado $\rightarrow$ terminar anticipadamente.

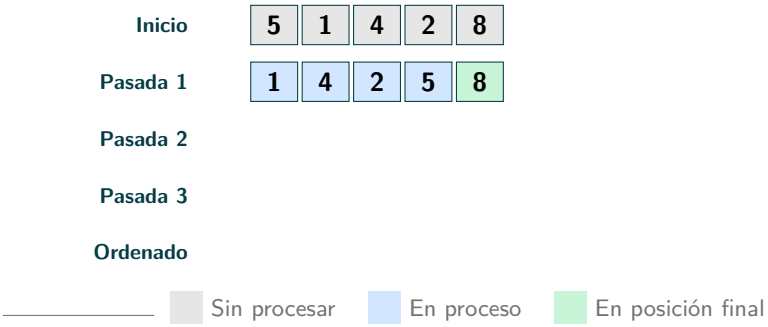
Implementación en C++

```
void burbuja(int arr[], int n) {
  for (int i = 0; i < n-1; ++i) {
    bool intercambio = false;
    for (int j = 0; j < n-1-i; ++j){
      if (arr[j] > arr[j+1]) {
        swap(arr[j], arr[j+1]);
        intercambio = true;
      }
    }
    // optimización: sin intercambios
    // → ya ordenado
    if (!intercambio) break;
  }
}
```

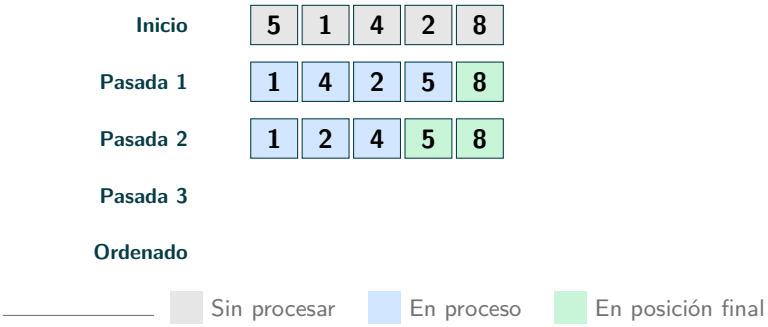
Bubble Sort — Traza completa: {5, 1, 4, 2, 8}



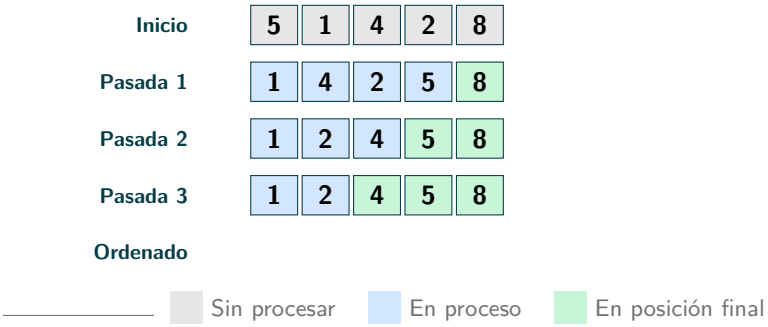
Bubble Sort — Traza completa: {5, 1, 4, 2, 8}



Bubble Sort — Traza completa: {5, 1, 4, 2, 8}



Bubble Sort — Traza completa: {5, 1, 4, 2, 8}



Bubble Sort — Traza completa: {5, 1, 4, 2, 8}



Bubble Sort — Análisis de comparaciones

Conteo de comparaciones para $N = 5$

Pasada	Comparaciones	Límite
1	4	$j < 4$
2	3	$j < 3$
3	2	$j < 2$
4	1	$j < 1$
Total	**10**	$\frac{N(N-1)}{2}$

$$T(N) = \frac{N(N-1)}{2} \in O(N^2)$$

En general para N elementos:

$$\underbrace{(N-1) + (N-2) + \cdots + 1}_{\text{suma aritmética}} = \frac{N(N-1)}{2}$$

Versión recursiva del burbuja

```
void burbujaRec(int arr[],int n){
  if (n <= 1) return; // base
  for (int j = 0; j < n-1; ++j)
    if (arr[j] > arr[j+1])
      swap(arr[j], arr[j+1]);
  // mayor ya está al final:
  // reducir el problema
  burbujaRec(arr, n - 1);
}
```

Ejercicios propuestos

1. Adaptar para orden **decreciente**, o para arreglos de string
2. Probar la versión recursiva.

Selection Sort — Idea central

Principio

En cada pasada i, **busca el mínimo** en el subarreglo $[\,i \ldots N{-}1\,]$ y lo **intercambia** con el elemento en la posición i.

- Siempre hace exactamente $\dfrac{N(N-1)}{2}$ **comparaciones** (sin optimización).
- Solo hace **a lo sumo** $N-1$ **intercambios**.
- El frente del arreglo crece como parte ya ordenada.

Complejidad: siempre $O(n^2)$ en comparaciones; pero mínimo de intercambios.

Implementación en C++

```
void seleccion(int arr[], int n) {
  for (int i = 0; i < n-1; ++i) {
    // Buscar índice del mínimo
    int indMin = i;
    for (int j = i+1; j < n; ++j) {
      if (arr[j] < arr[indMin])
        indMin = j;
    }
    // Interc. mínimo con pos i
    if (indMin != i)
      swap(arr[i], arr[indMin]);
  }
}
```

Selection Sort — Traza completa: {7, 1, 4, 2, 3}



Selection Sort — Traza completa: $\{7, 1, 4, 2, 3\}$



Selection Sort — Traza completa: $\{7, 1, 4, 2, 3\}$



Selection Sort — Traza completa: $\{7, 1, 4, 2, 3\}$



Selection Sort — Traza completa: {7, 1, 4, 2, 3}



Selection Sort — Traza completa: {7, 1, 4, 2, 3}



Selection Sort — Versión recursiva y ejercicios

```
// Encuentra índice del mínimo
int indiceMin(int arr[], int ini, int n) {
 if (ini == n-1) return ini;
 int iResto = indiceMin(arr, ini+1, n);
 return (arr[ini] <= arr[iResto])? ini :
  ↪  iResto;
}

void seleccionRec(int arr[],
                  int ini, int n) {
  if (ini >= n-1) return; // base
  int im = indiceMin(arr, ini, n);
  if (im != ini)
    swap(arr[ini], arr[im]);
  seleccionRec(arr, ini+1, n);
}
```

Comparación: burbuja vs. selección

	Burbuja	**Selección**
Comparaciones	$O(n^2)$	$O(n^2)$
Intercambios	$O(n^2)$	$O(n)$
Mejor caso	$O(n)^*$	$O(n^2)$
Estable	✔ Sí	✖ No

*Con la optimización de parada temprana.

Ejercicios propuestos

1. Adaptar para orden **decreciente**
2. Probar la versión recursiva.

Insertion Sort — Idea central

Principio

Funciona como **ordenar cartas en la mano**: se toma una carta nueva y se inserta en la posición correcta dentro de las ya ordenadas.

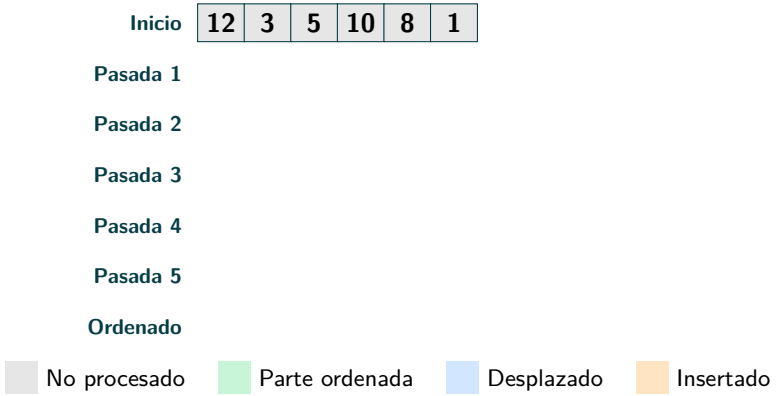
- `arr[0]` se considera ya ordenado.
- En la pasada i, se toma `arr[i]` como **clave** y se desplazan los elementos mayores una posición a la derecha hasta hallar su lugar.
- $N - 1$ pasadas para un arreglo de N elementos.

Complejidad: peor caso $O(n^2)$; **mejor caso** $O(n)$ (arreglo casi ordenado).

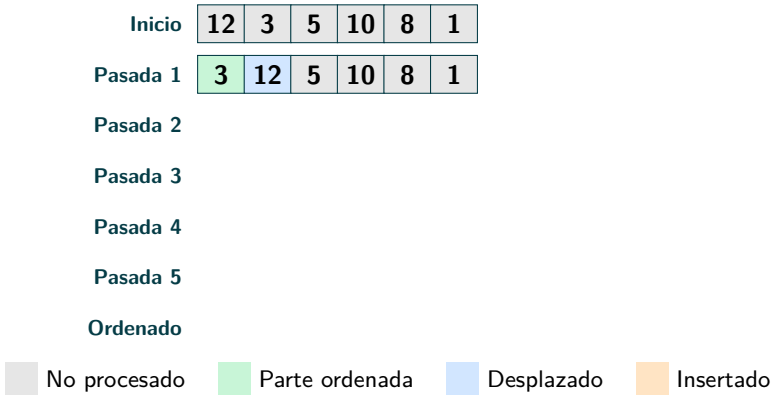
Implementación en C++

```
void insercion(int arr[], int n) {
  for (int i = 1; i < n; ++i) {
    int clave = arr[i];
    int j = i - 1;
    // Desplazar elementos mayores
    // que clave una posición
    while (j >= 0 &&
           arr[j] > clave) {
      arr[j+1] = arr[j];
      j--;
    }
    // Insertar clave en su lugar
    arr[j+1] = clave;
  }
}
```

Insertion Sort — Traza completa: {12, 3, 5, 10, 8, 1}



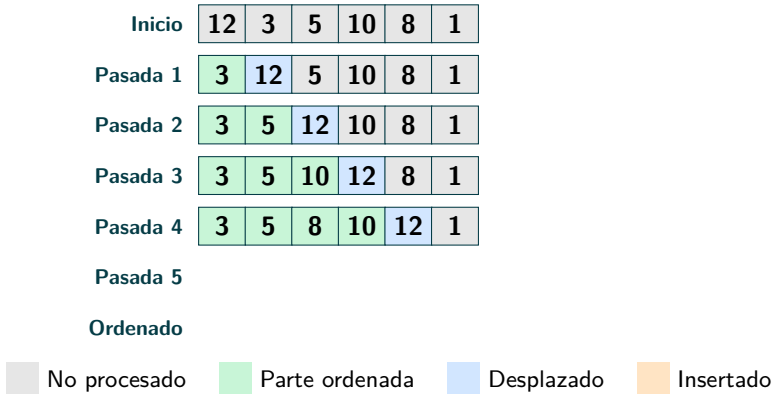
Insertion Sort — Traza completa: {12, 3, 5, 10, 8, 1}



Insertion Sort — Traza completa: {12, 3, 5, 10, 8, 1}



Insertion Sort — Traza completa: {12, 3, 5, 10, 8, 1}



Insertion Sort — Traza completa: {12, 3, 5, 10, 8, 1}



Insertion Sort — Traza completa: {12, 3, 5, 10, 8, 1}



Insertion Sort — Desplazamiento vs. intercambio

¿Por qué desplazar y no intercambiar?

Insertion Sort **desplaza** los elementos en lugar de intercambiarlos. Esto reduce las operaciones de escritura a la mitad.

Insertar 3 en {12, ?, ?, ...}



Versión recursiva

```
void insercionRec(int arr[],int n) {
  if (n <= 1) return; // base
  // Ordenar los primeros n-1
  insercionRec(arr, n-1);
  // Insertar arr[n-1] en su lugar
  int ultimo = arr[n-1];
  int j = n-2;
  while (j >= 0 && arr[j] > ultimo) {
    arr[j+1] = arr[j];
    j--;
  }
  arr[j+1] = ultimo;
}
```

Búsqueda Lineal — Concepto y código

Principio

Recorre el arreglo **elemento por elemento** comparando cada uno con el valor buscado (*clave*). Se detiene al encontrarlo o al llegar al final.

- Funciona con arreglos **desordenados**.
- Complejidad: **mejor caso** $O(1)$ (primer elemento), **peor caso** $O(n)$ (último o no existe).

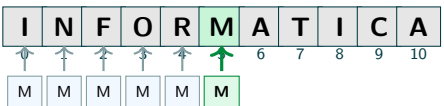
Cuándo usarla: arreglos pequeños, desordenados o cuando se busca una sola vez.

```
// Retorna índice o -1 si no existe
int busqLineal(int arr[], int n,
               int clave) {
  for (int i = 0; i < n; ++i)
    if (arr[i] == clave)
      return i;
  return -1;
}

// Versión que cuenta ocurrencias
int contarOcurrencias(int arr[],
           int n, int clave) {
  int cuenta = 0;
  for (int i = 0; i < n; ++i)
    if (arr[i] == clave) cuenta++;
  return cuenta;
}
```

Búsqueda Lineal — Traza y versión recursiva

Traza: buscar **M** en:
{I, N, F, O, R, M, A, T, I, C, A}



Versión recursiva

```
int busqLinealRec(int arr[], int n,
                  int clave,
                  int ini = 0) {
  if (ini >= n) return -1;  // base
  if (arr[ini] == clave) return
  ↪  ini;
  return busqLinealRec(arr, n,
                       clave,
                       ↪  ini+1);
}
```

Generación de arreglos aleatorios para pruebas

Enteros aleatorios

```
#include <cstdlib>
#include <ctime>

void generarEnteros(int arr[],
                    int n,
                    int min, int
                    ↪  max){
  srand(time(0)); // semilla
  for (int i = 0; i < n; ++i)
    arr[i] = min +
             rand() % (max-min+1);
}
// generarEnteros(a, 10, 1, 100)
```

```
const int N = 10; int arr[N];
  generarEnteros(arr, N, 1, 99);
  imprimirArr(arr, N, "Original");
  int op;
  cout << "1.Burbuja 2.Selección "
       << "3.Inserción: ";
  cin >> op;
  if      (op==1) burbuja(arr,N);
  else if (op==2) seleccion(arr,N);
  else if (op==3) insercion(arr,N);
  imprimirArr(arr, N, "Ordenado");
  int clave;
  cout << "Buscar: "; cin >> clave;
  int idx = busqLineal(arr,N,clave);
  if (idx >= 0)
    cout << "Índice: " << idx;
  else cout << "No encontrado\n";
```

Actividad práctica — Sesión 2

1. Implementar los tres algoritmos de ordenamiento con funciones separadas. Probar con el arreglo $\{64, 34, 25, 12, 22, 11, 90\}$.
2. Añadir a **burbuja** la optimización de parada temprana. Verificar con un arreglo ya ordenado.
3. Implementar **búsqueda lineal** para: (a) primera ocurrencia, (b) última ocurrencia, (c) contar ocurrencias.
4. Adaptar los tres algoritmos para **cadenas** (`string`). Ordenar nombres alfabéticamente.
5. Adaptar para **orden decreciente**: ¿qué operador cambia?

Adicionales

6. Escribir las versiones **recursivas** de los tres algoritmos de ordenamiento.
7. Generar un arreglo de **doubles** aleatorios en $[0, 1]$ (tamaño 20), ordenarlo con inserción e imprimirlo con 4 decimales.
8. Implementar un **menú interactivo** que permita: generar arreglo, elegir algoritmo de ordenamiento, buscar un elemento.
9. **Contar** cuántas comparaciones e intercambios realiza cada algoritmo para el mismo arreglo de entrada. Comparar los resultados.

Próxima sesión

Algoritmos de Ordenamiento y Búsqueda II

Merge Sort, Quick Sort y Búsqueda Binaria

achulluncuy@uni.edu.pe

Repositorio del curso

Aula virtual UNI

"Sorting is the central problem in computer science; once you can sort, you can do everything else."

— Donald Knuth, *The Art of Computer Programming*

